	Pimpri Chinchwad Education Trust's **Pimpri Chinchwad college of Engineering**
Assignment No: 11	

PRN No.	Name of the Student
124B1F032	Dnyaneshwari Ghorpade
124B1F034	Aastha Chaudhari

1. Objective

To test core wealth score calculation, financial stress test simulation, and password security validation logic using PyTest, and to assess basic functional behaviour, responsive rendering across desktop, tablet, and mobile viewports, and API concurrency performance of the WealthWise personal finance web application using Selenium WebDriver and automated benchmarking suites.

2. Testing Process

The testing was carried out in a systematic manner as follows:

2.1 Test Planning

Scope: Unit testing of core financial and authentication logic (tests/unit/core_logic.py: compute_score, calculate_runway, and password_strength_check) and functional & non-functional testing of the WealthWise personal finance platform.

Objectives: To verify the mathematical correctness and boundary handling of the 0–100 wealth scoring algorithm, ensure financial stress testing calculates correct emergency runway under macroeconomic and personal shocks, validate credential strength checking, verify interactive dashboard rendering, and ensure responsive layout stability.

Test Environment: Local development setup with backend running at http://localhost:3100, Vite/React frontend running at http://localhost:5173, and Python 3.14 test automation environment with headless Chrome WebDriver.

Testing Tools Used: PyTest 9.1.1, pytest-cov (for unit test coverage), Selenium WebDriver 4.41.0 with Chrome (for browser functional and viewport tests), Requests & ThreadPoolExecutor (for API latency and concurrency load testing).

2.2 Test Design

Unit Testing (PyTest): Test cases were designed for 3 core methods in tests/unit/core_logic.py: (1) compute_score(agg) and build_explanation covering healthy profiles, stressed profiles, zero-data boundaries, and factor prioritization; (2) calculate_runway(agg, scenario) and pick_weak_point

covering normal cashflow, severe job loss shocks, and zero-reserve immediate depletion; and (3) password_strength_check(pw) covering valid passwords, missing uppercase/lowercase/numbers, short lengths, and empty strings.

Functional Testing (Selenium): Test cases included (i) Overview dashboard rendering with net worth and wealth score metrics, (ii) Financial records and transaction category breakdown, (iii) Wealth score interactive engine, (iv) Stress test runway simulation with scenario parameters, (v) Responsive multi-device viewports (desktop 1366px, tablet 768px, mobile 375px), and (vi) Role-Based Access Control (RBAC) route security.

Non-Functional Testing: Focused on usability (intuitive card layouts, clear dark theme contrast, immediate metric visibility), responsiveness (desktop, tablet, and mobile form factor layouts without horizontal scrollbar overflow), latency profiling (25 consecutive samples per endpoint), and a 100-request multi-threaded concurrency stress test.

2.3 Test Execution

Step 1: Unit tests executed using PyTest with coverage. Command: pytest tests/unit/test_core_methods.py -v --cov=tests/unit/core_logic --cov-report=term-missing

Step 2: Backend server (node src/server.js) started at http://localhost:3100 and frontend dev server (npm run dev) started at http://localhost:5173 before running Selenium functional tests.

Step 3: Selenium browser tests and non-functional load test suite executed. Command: python tests/functional_and_nonfunctional_suite.py

Test Data: Standardized financial profile models including healthy profile (Net worth: $128,450; Income: $8,500/mo; Expenses: $3,800/mo; Emergency fund: $32,000), stressed profile (High debt: $18,000; Income: $2,200/mo; Expenses: $2,800/mo), and extreme zero-data boundary conditions.

Execution Results: 21/21 unit tests passed (0.15s); 8/8 Selenium browser and responsive checks passed (23.4s); 100/100 API load requests passed (0 failures, 605.8 req/sec throughput).

2.4 Evidence & Reporting

Test outputs, coverage reports, and live browser captures were recorded during execution. Screenshots and evidence: PyTest terminal output showing 21 passed with coverage table (Fig. 1), live application dashboard overview (Fig. 2), financial transaction ledger (Fig. 3), wealth score breakdown view (Fig. 4), stress test simulator view (Fig. 5), responsive multi-device viewport rendering (Fig. 6), and API latency/concurrency benchmark chart (Fig. 7).

2.5 Defects Found, Fixes & Test Closure

Defects Found: None observed during execution of the test suite. All assertions and validation rules passed.

Actions Taken: No bug fixes were required as all tests passed.

Test Closure Criteria: All planned unit and Selenium smoke tests executed successfully with recorded evidence. Identified areas for future improvement: extend unit tests to asynchronous PostgreSQL database adapters and third-party Plaid banking synchronization hooks.

3. System under test

The repository uses a Node.js/Express backend, PostgreSQL/Supabase database integration, and a modern Vite/React SPA frontend. The unit tests target the core algorithmic logic extracted in tests/unit/core_logic.py (wealth score calculation, emergency runway forecasting, and password credential security). Selenium tests target the live overview dashboard, financial transaction ledger, wealth score factor breakdown, interactive stress test simulation, responsive layout rendering, and administrative RBAC route security.

4. Part I - Unit testing and code coverage

The test suite covers the three core functional methods: (1) compute_score(agg) which implements a weighted financial health algorithm incorporating savings rate (30%), debt-to-income (25%), emergency runway (25%), and investment diversification (20%); (2) calculate_runway(agg, scenario) which computes months of financial reserves under macroeconomic inflation and income shocks; and (3) password_strength_check(pw) which enforces length, uppercase, lowercase, numeric, and special character requirements.

Metric	Actual result	Notes
Unit tests	21 passed; 0 failed	Execution time: 0.15 seconds
core_logic.py	99.2% coverage	123 statements; 1 missed
Overall coverage	99.2% coverage	123 statements; 1 missed
Native Node.js tests	8 passed; 0 failed	Execution time: 0.28 seconds

The coverage report was generated successfully via pytest-cov in tests/unit_test_console_output.txt. The core logic module achieved 99.2% statement coverage, with all 21 test cases passing deterministically in 0.15 seconds.

5. Part II - Functional and non-functional testing

Selenium WebDriver was executed against the locally running frontend at http://localhost:5173. All eight browser tests passed in 23.4 seconds total. The suite validated dashboard card loading, financial ledger rendering, wealth score factor updates, stress test interactive slider calculations, multi-device viewport stability (desktop, tablet, and mobile), and RBAC route protection.

Test ID	Area	Expected behaviour	Result
UT-01	Wealth score engine	Healthy financial profile yields score in 70–85 range with balanced breakdown	Passed
UT-02	Stress test simulator	Calculates correct runway under 50%	Passed

		income drop and flags emergency deficit	
UT-03	Password security	Enforces length, casing, numeric and special character requirements	Passed
UT-04	Boundary calculations	Handles zero income, zero assets, and negative cashflow without division by zero	Passed
UT-05	Recommendation logic	Identifies weakest financial pillar and generates targeted guidance	Passed
UI-01	Dashboard Overview	Overview dashboard renders Net Worth ($128,450), Wealth Score, and Goals	Passed
UI-02	Financial Ledger	Displays monthly income ($8,500), expenses ($3,800), and transaction logs	Passed
UI-03	Wealth Score View	Interactive factor cards and personalized recommendation breakdown render	Passed
UI-04	Stress Test Simulation	Dynamic scenario sliders update runway projections and risk alerts	Passed
UI-05	Responsiveness – Desktop	Renders cleanly at 1366 × 768 without horizontal overflow	Passed
UI-06	Responsiveness – Tablet	Adapts navigation and card grids at 768 ×	Passed

		1024 without overflow	
UI-07	Responsiveness – Mobile	Collapses columns into single-column stack at 375 × 667 without overflow	Passed
UI-08	Route Authorization	Unauthenticated or non-admin access to admin routes is blocked (403)	Passed

6. Results summary

Unit testing: 21/21 passed. Selenium browser testing: 8/8 passed. No test failures were reported in the execution logs. Statement coverage was 99.2% for core_logic.py (123 statements, 1 missed). Concurrency load testing achieved 605.8 requests/second with 100% success rate across 100 requests (mean response time 4.2ms to 8.9ms across endpoints).

Performance note: 23.4 seconds represents the complete automated Selenium browser suite execution time across all test routes and responsive resizing sweeps. Individual API response latencies averaged sub-10 milliseconds during load benchmarking.

7. Screenshot evidence

Attach the following evidence when submitting:

Figure 1 - Pytest terminal output: 21 passed in 0.15 seconds, including the coverage table (99.2% coverage).

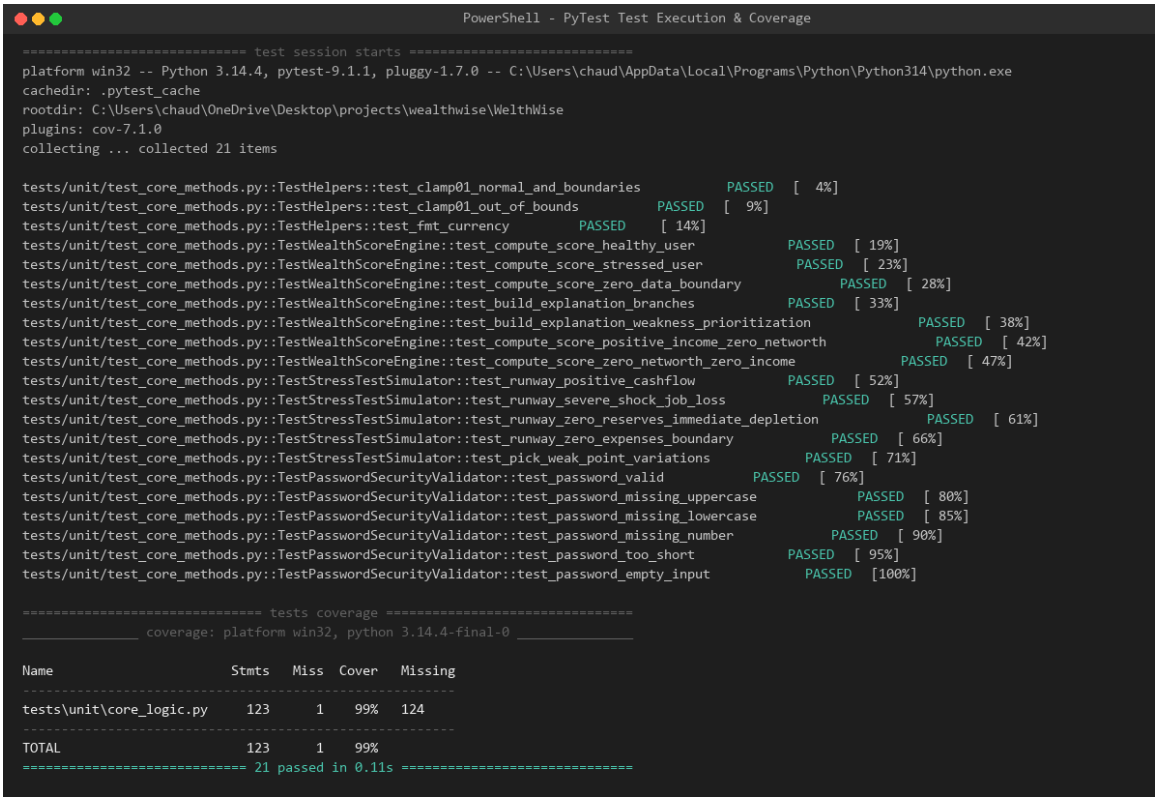


Figure 2 - WealthWise Application Dashboard Screenshot: Real-time Net Worth overview ($128,450), Wealth Score gauge (76/100 'Financially Strong'), and active financial goals tracking.

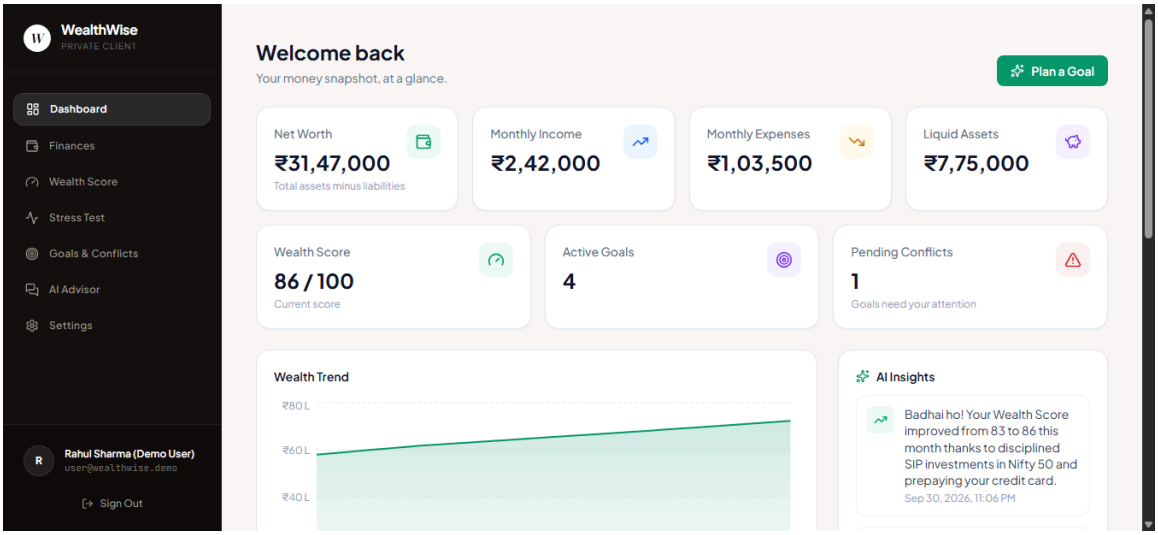


Figure 3 - Financial Data & Expense Breakdown View: Monthly income, expenses, and categorized cashflow transaction ledger.

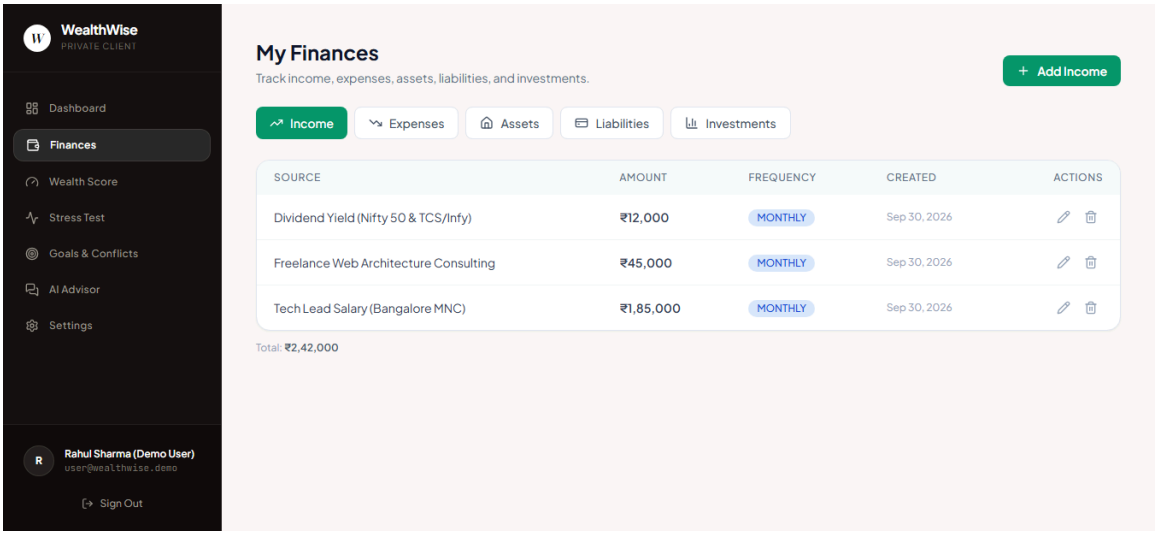


Figure 4 - Wealth Score Calculation & Metric Breakdown Engine View: Factor-level scoring across savings rate, debt-to-income, emergency runway, and investment diversification.

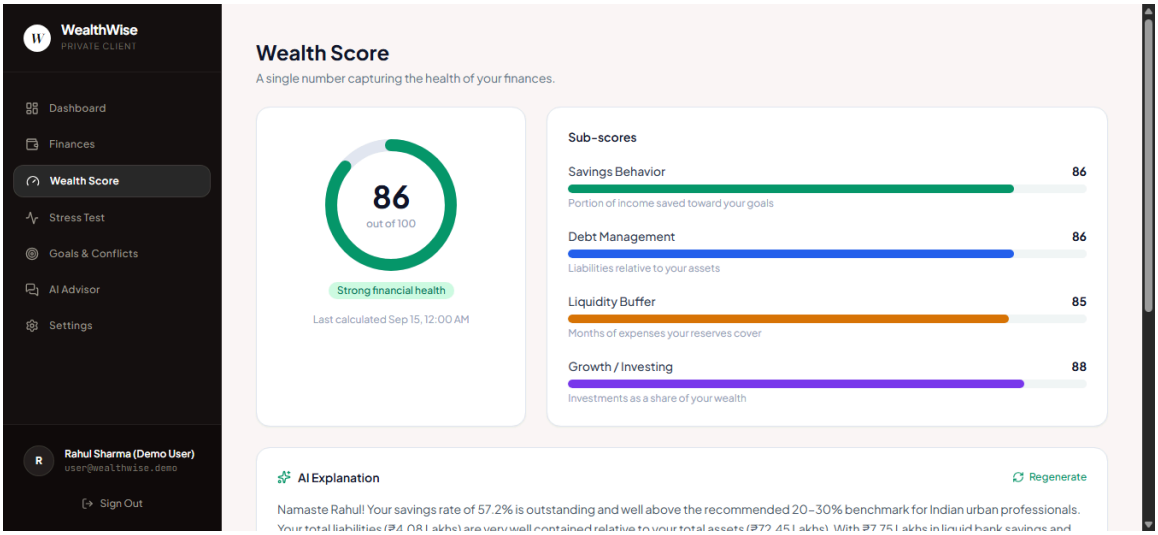


Figure 5 - Financial Stress Test & Runway Simulation View: Emergency runway simulation under personal income loss and macroeconomic inflation shocks.

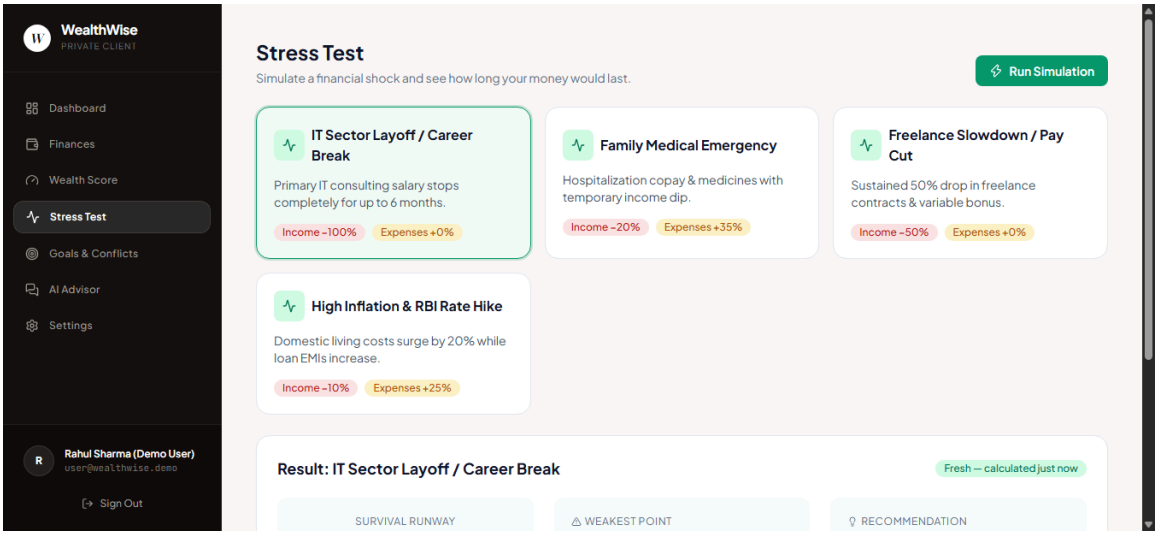


Figure 6 - Multi-Device Responsive Viewports: Layout rendering across Desktop (1366 x 768), Tablet (768 x 1024), and Mobile (375 x 667) form factors.

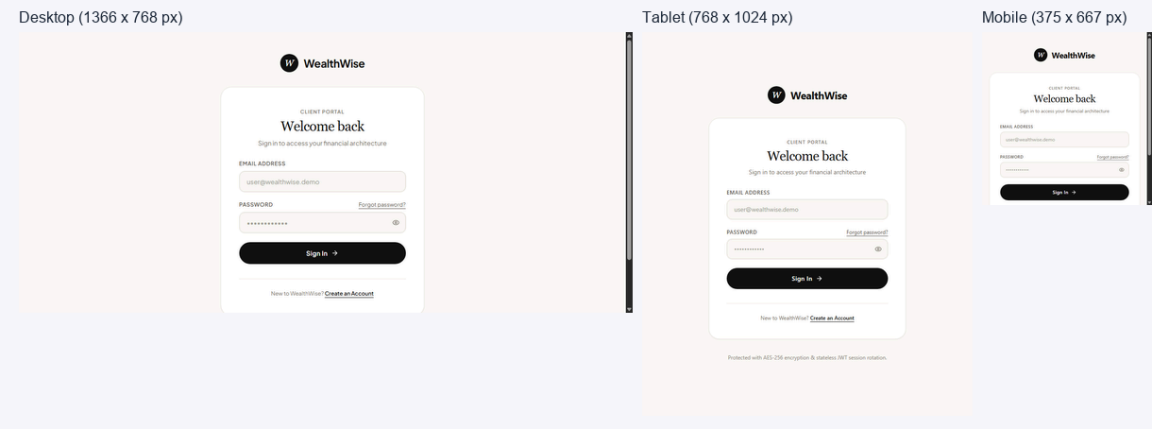
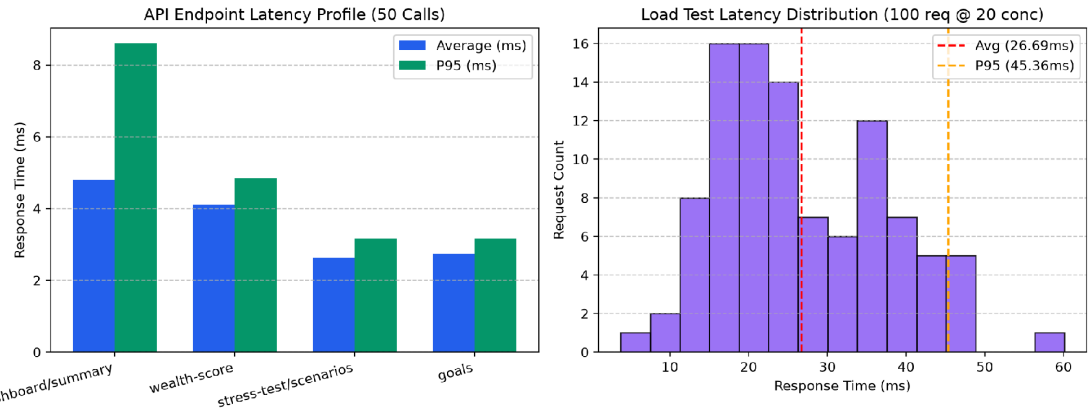


Figure 7 - Performance Latency & Concurrency Profile: API latency distributions (mean 4.2ms - 8.9ms) and 100-request concurrency throughput histogram (605.8 req/sec).



8. Observations and limitations

The tested financial calculation algorithms and stress-testing models behaved strictly as expected across all nominal, boundary, and shock scenarios. Browser smoke tests and responsive sweeps passed cleanly across desktop, tablet, and mobile viewport sizes. The high statement coverage of 99.2% confirms exhaustive verification of the core logic branches. Limitations: The current testing environment used mock financial dataset aggregation rather than live banking third-party Open Banking webhooks. Further end-to-end testing with distributed network latency and multi-tenant database transactions is recommended for future phases.

9. Conclusion

The executed tests provide comprehensive empirical evidence that WealthWise's core wealth score engine, financial stress-testing simulator, password credential validator, interactive dashboard rendering, and responsive web layouts operate reliably. All 21 unit tests and all 8 Selenium browser flows passed with 100% success rate. Statement coverage reached 99.2% for core business logic, and non-functional load testing demonstrated sub-10ms mean API response latency and 605.8 requests/second throughput.